Lab 7: Simulated Annealing

To implement the Simulated Annealing algorithm for finding the maximum value of

$$g(x) = \sin(x) + \frac{\sin(3x)}{3}$$

Step 1: Import the modules

```
import math, random
```

math → used for sin() and exp()

random → used to choose a random neighbour and make the acceptance decision

Step 2: Define objective function

```
def g(x):
    return math.sin(x) + math.sin(3*x) / 3
```

For any value of x the function calculates g(x).

Step 3: Declare function

```
def simulated_annealing(start, T=1.5, cool=0.82, T_min=0.03):
    random.seed(2)
    x = start
```

start: starting value of x

T: starting temperature

cool: cooling factor

T_min: temperature at which we stop

random.seed(): makes random sequence same every time we run

Step 4: Loop on temperature

```
while T > T_min:
```

T shrinks after every round. This is important to escape the local maxima

classmate
Date
Page

Step 5: Pick a random neighbour

```
candidate = x + random.uniform(-1, 1)
candidate = max(0.0, min(6.0, candidate))
```

random.uniform(-1, 1) picks a random number between -1 & 1.
candidate creates one random neighbour.
The second line keeps candidate between 0.0 and 6.

Step 6: Measure the change in score

```
dE = g(candidate) - g(x)
```

dE tells whether the new position is better or worse

Step 7: Always take improvement

```
if dE > 0:
    accept = True
```

If the new position gives a higher score we always take it

Step 8: Sometimes accept worse moves

```
else:
    p = math.exp(dE / T)
    accept = random.random() < p
```

If the move is worse we don't reject it automatically.
we calculate 'P' probability of taking worse move

$$P = e^{dE/T}$$

Hight temperature → worse moves are likely accepted
Low temperature → worse moves are rarely accepted

Step 9: Move if accepted

```
if accept:
    x = candidate
```

If accept = True we move to the candidate.
If accept = False we stay at current x.
The move can happen in 2 ways
- candidate is better
- candidate is worse but randomly accepted.

Step 10: Cool down
T = T * cool
Since cool = 0.82, the temperature becomes
1.50 → 1.23 → 1.01 → ...
So the algorithm slowly changes from explore to optimize
At high temperature, more freedom to move downhill
At low temperature, mostly moves uphill.

Step 11: Return the final position
return x, g(x)

Step 12: Run and print
print ("SA from x=4.75 :", simulated_annealing(4.75))
run it from the exact point hill climbing failed